

Supplemental: Formalized Lean 4 Proofs Contexts and Pseudocontexts

Mirko Navara 

Karl Svozil 

June 2, 2026

Overview

This document gives the complete Lean 4 source for the formal verification accompanying the manuscript *Contexts and pseudocontexts*. The Lean development formalizes the finite coordinate-operator core of the paper: rank-one projectors, finite structure operators, the complex phase construction, ray-disjointness, the revised genuineness certificate, and the basic trace identities for rank-one decompositions.

The formalization is intentionally scoped to the algebraic spine of the article. It does not attempt to formalize the full projection-lattice interface, Gleason-theorem infrastructure, Schur–Horn theory, or the topological genericity arguments used elsewhere in the manuscript. Instead, it supplies a machine-checked certificate for the explicit finite-dimensional operator calculations that are most directly amenable to Lean verification.

The source has been checked with Lean 4 v4.30.0 and mathlib v4.30.0. In the archived workspace used for this supplement, the verification command is

```
powershell.exe -ExecutionPolicy Bypass -File .\verify.ps1.
```

The Lean files contain no placeholders such as `sorry` or `admit`.

The formalization consists of a single main Lean file:

- Section 2 (`Pseudocontexts/Basic.lean`) defines coordinate rank-one operators, structure operators, the root-of-unity phase construction, same-ray predicates, the revised plane-genuineness certificate, and trace identities.

Contents

1	Project Configuration Files	3
2	Formalized Pseudocontext Algebra	5

1 Project Configuration Files

lean-toolchain

```
leanprover/lean4:v4.30.0
```

lakefile.lean

```
1 import Lake
2 open Lake DSL
3
4 package Pseudocontexts where
5
6 require batteries from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/batteries"
7 require Qq from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/Qq"
8 require aesop from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/aesop"
9 require Cli from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/Cli"
10 require proofwidgets from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/proofwidgets"
11 require importGraph from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/importGraph"
12 require LeanSearchClient from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/LeanSearchClient"
13 require plausible from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/plausible"
14 require mathlib from "C:/Users/a/Documents/Codex/2026-05-30/can-you-help-me-with-lean/.lake/packages/mathlib"
15
16 @[default_target]
17 lean_lib Pseudocontexts where
```

Pseudocontexts.lean

```
1 import Pseudocontexts.Basic
```

verify.ps1

```
$ErrorActionPreference = "Stop"

$cache = "C:\Users\A\Documents\Codex\2026-05-30\can-you-help-me-with-lean\.lake\packages"
```

```
$lean = "C:\Users\A\elan\toolchains\leanprover--lean4---v4.30.0\bin\lean.exe"
$file = Join-Path $PSScriptRoot "Pseudocontexts\Basic.lean"

$paths = Get-ChildItem -Directory $cache |
  ForEach-Object { Join-Path $_.FullName ".lake\build\lib\lean" } |
  Where-Object { Test-Path $_ }

$env:LEAN_PATH = ($paths -join ";")
& $lean $file
```

2 Formalized Pseudocontext Algebra

Pseudocontexts/Basic.lean

```

1  import Mathlib.Data.Complex.Basic
2  import Mathlib.Data.Fintype.Basic
3  import Mathlib.Algebra.BigOperators.Group.Finset.Basic
4  import Mathlib.Algebra.BigOperators.Ring.Finset
5  import Mathlib.Data.Matrix.Mul
6  import Mathlib.Tactic.FinCases
7  import Mathlib.Tactic.Ring
8
9  open scoped BigOperators
10
11 namespace Pseudocontexts
12
13 set_option linter.unusedSectionVars false
14 set_option linter.unusedSimpArgs false
15 set_option linter.unnecessarySimp false
16
17 /-!
18 # Formal algebra for pseudocontexts
19
20 This file formalizes the operator-level algebra behind the paper's
21 pseudocontext proofs. We work with finite coordinate vectors over  $\mathbb{C}$ 
22 and rank-one matrices  $|v\rangle\langle v|$  represented as functions.
23 -/
24
25 abbrev CMatrix (n : Type) := n → n → ℂ
26
27 variable ι { n : Type} [Fintype ι] [DecidableEq ι]
28   [Fintype n] [DecidableEq n]
29
30 /-- The rank-one operator  $|v\rangle\langle v|$ , in coordinates. -/
31 def rankOne (v : n → ℂ) : CMatrix n :=
32   fun p q => v p * star (v q)
33
34 /-- The structure operator of a finite tuple of vectors. -/
35 def structureOp (a : ι → n → ℂ) : CMatrix n :=
36   fun p q => ∑ i : ι, rankOne (a i) p q
37
38 /-- Equality of rank-one sums is exactly equality of all matrix entries. -/
39 theorem structure_ext {a b : ι → n → ℂ}
40   (h : ∀ p q, ∑( i : ι, rankOne (a i) p q)
41     = ∑( i : ι, rankOne (b i) p q)) :
42   structureOp a = structureOp b := by
43     funext p q
44     exact h p q
45

```

```

46 /-- Two vectors span the same complex ray when one is a nonzero scalar
47 multiple of the other. -/
48 def SameRay (u v : n → ℂ) : Prop := ∃
49   z : ℂ, z ≠ 0 ∧ ∀ p, v p = z * u p
50
51 namespace PhaseConstruction
52
53 /-- The vectors `a_j = c 1e + s ω^j ze` in ℂ2. -/
54 def avec (c s ω : ℂ) (j : ℕ) : Fin 2 → ℂ
55   | 0 => c
56   | 1 => s * ω ^ j
57
58 /-- The phase-shifted vectors `b_j = c 1e + s ηω ^j ze` in ℂ2. -/
59 def bvec (c s ηω : ℂ) (j : ℕ) : Fin 2 → ℂ
60   | 0 => c
61   | 1 => s * η( * ω ^ j)
62
63 /-- Coordinate inner product on the two-dimensional support used by the
64 phase construction. -/
65 def inner2 (u v : Fin 2 → ℂ) : ℂ := ∑
66   p : Fin 2, star (u p) * v p
67
68 /-- A tuple is pairwise nonorthogonal when no two distinct entries are
69 orthogonal. This is the formal version of the  $\alpha \neq \pi/4$  branch in the
70 paper's phase construction. -/
71 def PairwiseNonorthogonal {k : ℕ} (a : Fin k → Fin 2 → ℂ) : Prop := ∀
72   j l : Fin k, j ≠ l → inner2 (a j) (a l) ≠ 0
73
74 /-- In the fixed two-dimensional support of the phase construction, a
75 context has exactly two rays. This records the cardinality obstruction
76 used in the revised paper. -/
77 def PlaneContextCardinality (k : ℕ) : Prop :=
78   k = 2
79
80 /-- A light-weight certificate that a tuple is not a context in the
81 two-dimensional support: either its cardinality is not two, or it is
82 pairwise nonorthogonal. -/
83 def NotContextInPlane {k : ℕ} (a : Fin k → Fin 2 → ℂ) : Prop :=
84   ¬ PlaneContextCardinality k ∨ PairwiseNonorthogonal a
85
86 /-- The revised genuineness certificate for the complex phase construction:
87 both tuples fail to be contexts in their common two-dimensional support. -/
88 def PhaseGenuineInPlane (c s ηω : ℂ) (k : ℕ) : Prop :=
89   NotContextInPlane (fun j : Fin k => avec c s ω (j : ℕ)) ∧
90   NotContextInPlane (fun j : Fin k => bvec c s ηω (j : ℕ))
91
92 theorem not_planeContextCardinality_of_three {k : ℕ} (hk : 3 ≤ k) :
93   ¬ PlaneContextCardinality k := by
94   intro h
95   unfold PlaneContextCardinality at h

```

```

96   have hbad :  $3 \leq 2$  := by simpa [h] using hk
97   exact (by decide :  $\neg 3 \leq 2$ ) hbad
98
99   /- This is the Lean-level counterpart of the revised paper statement
100   ``genuine whenever `k ≥ 3` or  $\alpha \neq \pi/4$ ``. The second branch is phrased
101   as the pairwise-nonorthogonality certificate produced by the
102   inner-product computation in the paper. -/
103   theorem phase_genuine_in_plane_of_three_or_pairwise {k : ℕ} {c s ηω : ℂ}
104     (h :  $3 \leq k$  ∨
105       (PairwiseNonorthogonal (fun j : Fin k => avec c s ω (j : ℕ)) ∧
106         PairwiseNonorthogonal (fun j : Fin k => bvec c s ηω (j : ℕ)))) :
107     PhaseGenuineInPlane c s ηω k := by
108   rcases h with hk | hpair
109   · constructor
110   · exact Or.inl (not_planeContextCardinality_of_three hk)
111   · exact Or.inl (not_planeContextCardinality_of_three hk)
112   · exact (Or.inr hpair.1, Or.inr hpair).2
113
114   theorem sum_rankOne_avec_apply_zero_zero (c s ω : ℂ) (k : ℕ) : ∑
115     (j : Fin k, rankOne (avec c s ω (j : ℕ)) 0 0)
116     = k • (c * star c) := by
117   simp [rankOne, avec]
118
119   theorem sum_rankOne_bvec_apply_zero_zero (c s ηω : ℂ) (k : ℕ) : ∑
120     (j : Fin k, rankOne (bvec c s ηω (j : ℕ)) 0 0)
121     = k • (c * star c) := by
122   simp [rankOne, bvec]
123
124   theorem sum_rankOne_avec_apply_zero_one {c s ω : ℂ} {k : ℕ}
125     (whstar : ∑(j : Fin k, star ω( ^ (j : ℕ))) = 0) : ∑
126     (j : Fin k, rankOne (avec c s ω (j : ℕ)) 0 1) = 0 := by
127   calc ∑
128     (j : Fin k, rankOne (avec c s ω (j : ℕ)) 0 1)
129     = ∑ j : Fin k, c * star s * star ω( ^ (j : ℕ)) := by
130   simp [rankOne, avec, mul_assoc]
131   _ = c * star s * ∑(j : Fin k, star ω( ^ (j : ℕ))) := by
132   rw ← [Finset.mul_sum]
133   _ = 0 := by rw [whstar]; simp
134
135   theorem sum_rankOne_bvec_apply_zero_one {c s ηω : ℂ} {k : ℕ}
136     (whstar : ∑(j : Fin k, star ω( ^ (j : ℕ))) = 0) : ∑
137     (j : Fin k, rankOne (bvec c s ηω (j : ℕ)) 0 1) = 0 := by
138   calc ∑
139     (j : Fin k, rankOne (bvec c s ηω (j : ℕ)) 0 1)
140     = ∑ j : Fin k, c * star s * star η * star ω( ^ (j : ℕ)) := by
141   simp [rankOne, bvec, mul_assoc]
142   _ = c * star s * star η * ∑(j : Fin k, star ω( ^ (j : ℕ))) := by
143   rw ← [Finset.mul_sum]
144   _ = 0 := by rw [whstar]; simp
145

```

```

146 theorem sum_rankOne_avec_apply_one_zero {c s ω : ℂ} {k : ℕ}
147   (ωh : ∑( j : Fin k, ω ^ (j : ℕ)) = 0) : ∑
148   ( j : Fin k, rankOne (avec c s ω (j : ℕ)) 1 0) = 0 := by
149   calc ∑
150     ( j : Fin k, rankOne (avec c s ω (j : ℕ)) 1 0)
151     = ∑ j : Fin k, s * star c * ω ^ (j : ℕ) := by
152       simp [rankOne, avec, mul_assoc, mul_comm, mul_left_comm]
153   _ = s * star c * ∑( j : Fin k, ω ^ (j : ℕ)) := by
154     rw <[ Finset.mul_sum]
155   _ = 0 := by rw [ωh]; simp
156
157 theorem sum_rankOne_bvec_apply_one_zero {c s ηω : ℂ} {k : ℕ}
158   (ωh : ∑( j : Fin k, ω ^ (j : ℕ)) = 0) : ∑
159   ( j : Fin k, rankOne (bvec c s ηω (j : ℕ)) 1 0) = 0 := by
160   calc ∑
161     ( j : Fin k, rankOne (bvec c s ηω (j : ℕ)) 1 0)
162     = ∑ j : Fin k, s * star c * η * ω ^ (j : ℕ) := by
163       simp [rankOne, bvec, mul_assoc, mul_comm, mul_left_comm]
164   _ = s * star c * η * ∑( j : Fin k, ω ^ (j : ℕ)) := by
165     rw <[ Finset.mul_sum]
166   _ = 0 := by rw [ωh]; simp
167
168 theorem sum_rankOne_avec_apply_one_one {c s ω : ℂ} {k : ℕ}
169   (ωhnorm : ∀ j : Fin k, ω ^ (j : ℕ) * star ω( ^ (j : ℕ)) = 1) : ∑
170   ( j : Fin k, rankOne (avec c s ω (j : ℕ)) 1 1)
171   = k • (s * star s) := by
172   calc ∑
173     ( j : Fin k, rankOne (avec c s ω (j : ℕ)) 1 1)
174     = ∑ j : Fin k, s * star s * ω
175       ( ^ (j : ℕ) * star ω( ^ (j : ℕ))) := by
176       simp [rankOne, avec, mul_assoc, mul_comm, mul_left_comm]
177   _ = ∑ _j : Fin k, s * star s := by
178     refine Finset.sum_congr rfl ?_
179     intro j _
180     calc
181       s * star s * ω( ^ (j : ℕ) * star ω( ^ (j : ℕ)))
182       = s * star s * 1 := by rw [ωhnorm j]
183     _ = s * star s := by ring
184   _ = k • (s * star s) := by simp
185
186 theorem sum_rankOne_bvec_apply_one_one {c s ηω : ℂ} {k : ℕ}
187   (ηhnorm : η * star η = 1)
188   (ωhnorm : ∀ j : Fin k, ω ^ (j : ℕ) * star ω( ^ (j : ℕ)) = 1) : ∑
189   ( j : Fin k, rankOne (bvec c s ηω (j : ℕ)) 1 1)
190   = k • (s * star s) := by
191   calc ∑
192     ( j : Fin k, rankOne (bvec c s ηω (j : ℕ)) 1 1)
193     = ∑ j : Fin k, s * star s * η( * star η) * ω
194       ( ^ (j : ℕ) * star ω( ^ (j : ℕ))) := by
195       simp [rankOne, bvec, mul_assoc, mul_comm, mul_left_comm]

```



```

196   _ = ∑ _j : Fin k, s * star s := by
197     refine Finset.sum_congr rfl ?_
198     intro j _
199     calc
200       s * star s * η( * star η) * ω
201       ( ^ (j : ℕ) * star ω( ^ (j : ℕ)))
202       = s * star s * 1 * 1 := by rw [ηhnorm, ωhnorm j]
203   _ = s * star s := by ring
204   _ = k • (s * star s) := by simp
205
206 /-- Algebraic core of the complex-phase construction: a phase-shifted
207 root-of-unity tuple gives the same structure operator. -/
208 theorem phase_structure_eq {k : ℕ} {c s ηω : ℂ}
209   (wh : ∑( j : Fin k, ω ^ (j : ℕ)) = 0)
210   (whstar : ∑( j : Fin k, star ω( ^ (j : ℕ))) = 0)
211   (ηhnorm : η * star η = 1)
212   (ωhnorm : ∀ j : Fin k, ω ^ (j : ℕ) * star ω( ^ (j : ℕ)) = 1) :
213   structureOp (fun j : Fin k => avec c s ω (j : ℕ))
214   = structureOp (fun j : Fin k => bvec c s ηω (j : ℕ)) := by
215   funext p q
216   fin_cases p <=> fin_cases q
217   · simp [structureOp, sum_rankOne_avec_apply_zero_zero,
218     sum_rankOne_bvec_apply_zero_zero]
219   · simp [structureOp, sum_rankOne_avec_apply_zero_one whstar,
220     sum_rankOne_bvec_apply_zero_one whstar]
221   · simp [structureOp, sum_rankOne_avec_apply_one_zero wh,
222     sum_rankOne_bvec_apply_one_zero wh]
223   · simp [structureOp, sum_rankOne_avec_apply_one_one ωhnorm,
224     sum_rankOne_bvec_apply_one_one ηhnorm ωhnorm]
225
226 /-- If two `a`-vectors are on the same ray, the corresponding powers
227 of ω`` agree. -/
228 theorem sameRay_avec_imp_pow_eq {c s ω : ℂ} {j l : ℕ}
229   (hc : c ≠ 0) (hs : s ≠ 0)
230   (h : SameRay (avec c s ω j) (avec c s ω l)) : ω
231   ^ l = ω ^ j := by
232   rcases h with ⟨z, _hz, _hzv⟩
233   have hz : z = 1 := by
234     have h0 := hzv 0
235     have h0' : (1 : ℂ) * c = z * c := by simp [avec] using h0
236     exact (mul_right_cancel hc h0').symm
237   have h1 := hzv 1
238   have h1' : s * ω ^ l = s * ω ^ j := by simp [avec, hz] using h1
239   exact (mul_left_cancel hs h1')
240
241 /-- If two phase-shifted `b`-vectors are on the same ray, the
242 corresponding powers of ω`` agree. -/
243 theorem sameRay_bvec_imp_pow_eq {c s ηω : ℂ} {j l : ℕ}
244   (hc : c ≠ 0) (hs : s ≠ 0) (ηh : η ≠ 0)
245   (h : SameRay (bvec c s ηω j) (bvec c s ηω l)) : ω

```

```

246   ^ l = ω ^ j := by
247   rcases h with (z, _hz, )hzv
248   have hz : z = 1 := by
249     have h0 := hzv 0
250     have h0' : (1 : ℂ) * c = z * c := by simpa [bvec] using h0
251     exact (mul_right_cancel hc h0').symm
252   have h1 := hzv 1
253   have h1' : s * η * ω ^ l = s * η * ω ^ j := by
254     simpa [bvec, hz, mul_assoc] using h1
255   exact mul_left_cancel (mul_ne_zero hs ηh) h1'
256
257 /-- If an `a`-vector and a phase-shifted `b`-vector are on the same
258 ray, then the phase relation  $\eta^l \omega^l = \omega^j$  holds. -/
259 theorem sameRay_avec_bvec_imp_phase_eq {c s ηω : ℂ} {j l : ℕ}
260   (hc : c ≠ 0) (hs : s ≠ 0)
261   (h : SameRay (avec c s ω j) (bvec c s ηω l)) : η
262     * ω ^ l = ω ^ j := by
263   rcases h with (z, _hz, )hzv
264   have hz : z = 1 := by
265     have h0 := hzv 0
266     have h0' : (1 : ℂ) * c = z * c := by simpa [avec, bvec] using h0
267     exact (mul_right_cancel hc h0').symm
268   have h1 := hzv 1
269   have h1' : s * η( * ω ^ l) = s * ω ^ j := by
270     simpa [avec, bvec, hz, mul_assoc] using h1
271   exact mul_left_cancel hs h1'
272
273 /-- A compact formal pseudocontext statement for the complex-phase
274 construction. The hypotheses say exactly that the phases cancel in
275 the off-diagonal entries, have unit modulus on the diagonal, and do not
276 create ray collisions. -/
277 theorem complex_phase_pseudocontext {k : ℕ} {c s ηω : ℂ}
278   (hc : c ≠ 0) (hs : s ≠ 0)
279   (ωh : ∑( j : Fin k, ω ^ (j : ℕ)) = 0)
280   (ωhstar : ∑( j : Fin k, star ω( ^ (j : ℕ))) = 0)
281   (ηhnorm : η * star η = 1)
282   (ωhnorm : ∀ j : Fin k, ω ^ (j : ℕ) * star ω( ^ (j : ℕ)) = 1)
283   (ha_distinct : ∀ j l : Fin k, ω ^ (j : ℕ) = ω ^ (l : ℕ) → j = l)
284   (hcross : ∀ j l : Fin k, η * ω ^ (l : ℕ) ≠ ω ^ (j : ℕ)) :
285   structureOp (fun j : Fin k => avec c s ω (j : ℕ))
286     = structureOp (fun j : Fin k => bvec c s ηω (j : ℕ)) ∧
287     ( j l : Fin k,
288       SameRay (avec c s ω (j : ℕ)) (avec c s ω (l : ℕ)) → j = l) ∧
289     ( j l : Fin k,
290       SameRay (bvec c s ηω (j : ℕ)) (bvec c s ηω (l : ℕ)) → j = l) ∧
291     ( j l : Fin k,
292       ¬ SameRay (avec c s ω (j : ℕ)) (bvec c s ηω (l : ℕ))) := by
293   refine (phase_structure_eq ωh ωhstar ηhnorm ωhnorm, ?_, ?_, ?_)
294   · intro j l h
295     apply ha_distinct

```

```

296   exact (sameRay_avec_imp_pow_eq hc hs h).symm
297   · intro j l h
298     apply ha_distinct
299     have  $\eta h : \eta \neq 0 := \text{fun } \eta h0 \Rightarrow \text{by}$ 
300       have  $(0 : \mathbb{C}) = 1 := \text{by}$ 
301         simp [ηh0] using ηhnorm
302       exact zero_ne_one this
303     exact (sameRay_bvec_imp_pow_eq hc hs ηh h).symm
304   · intro j l h
305     exact hcross j l (sameRay_avec_bvec_imp_phase_eq hc hs h)
306
307   /- The complex phase construction together with the revised genuineness
308   certificate: the common-operator and ray-disjointness conclusions are the
309   same as in `complex_phase_pseudocontext`, and genuineness is certified by
310   either `k ≥ 3` or pairwise nonorthogonality of both tuples. -/
311   theorem complex_phase_pseudocontext_with_genuineness {k : ℕ} {c s ηω : ℂ}
312     (hc : c ≠ 0) (hs : s ≠ 0)
313     (ωh :  $\sum (j : \text{Fin } k, \omega ^ (j : \mathbb{N})) = 0$ )
314     (ωhstar :  $\sum (j : \text{Fin } k, \text{star } \omega ( ^ (j : \mathbb{N}))) = 0$ )
315     (ηhnorm :  $\eta * \text{star } \eta = 1$ )
316     (ωhnorm :  $\forall j : \text{Fin } k, \omega ^ (j : \mathbb{N}) * \text{star } \omega ( ^ (j : \mathbb{N})) = 1$ )
317     (ha_distinct :  $\forall j l : \text{Fin } k, \omega ^ (j : \mathbb{N}) = \omega ^ (l : \mathbb{N}) \rightarrow j = l$ )
318     (hcross :  $\forall j l : \text{Fin } k, \eta * \omega ^ (l : \mathbb{N}) \neq \omega ^ (j : \mathbb{N})$ )
319     (hgenuine :  $3 \leq k \vee$ 
320       (PairwiseNonorthogonal (fun j : Fin k => avec c s ω (j : ℕ)) ∧
321         PairwiseNonorthogonal (fun j : Fin k => bvec c s ηω (j : ℕ)))) :
322     (structureOp (fun j : Fin k => avec c s ω (j : ℕ))
323       = structureOp (fun j : Fin k => bvec c s ηω (j : ℕ)) ∧
324       (j l : Fin k,
325         SameRay (avec c s ω (j : ℕ)) (avec c s ω (l : ℕ)) → j = l) ∧
326       (j l : Fin k,
327         SameRay (bvec c s ηω (j : ℕ)) (bvec c s ηω (l : ℕ)) → j = l) ∧
328       (j l : Fin k,
329         ¬ SameRay (avec c s ω (j : ℕ)) (bvec c s ηω (l : ℕ)))) ∧
330     PhaseGenuineInPlane c s ηω k := by
331   exact (complex_phase_pseudocontext hc hs ωh ωhstar ηhnorm ωhnorm
332     ha_distinct hcross,
333     phase_genuine_in_plane_of_three_or_pairwise )hgenuine
334
335 end PhaseConstruction
336
337 namespace TraceIdentities
338
339 variable ι { n : Type} [Fintype ι] [DecidableEq ι]
340   [Fintype n] [DecidableEq n]
341
342 /- Coordinate inner product, conjugate-linear in the first variable. -/
343 def inner (u v : n → ℂ) : ℂ :=
344    $\sum p : n, \text{star } (u p) * v p$ 
345

```

```

346 /-- Matrix product for coordinate matrices. -/
347 def mmul (A B : CMatrix n) : CMatrix n :=
348   fun p q => ∑ r : n, A p r * B r q
349
350 /-- Coordinate trace. -/
351 def trace (A : CMatrix n) : ℂ := ∑
352   p : n, A p p
353
354 theorem trace_rankOne (v : n → ℂ) :
355   trace (rankOne v) = inner v v := by
356   simp [trace, inner, rankOne, mul_comm]
357
358 theorem trace_structure (a : ι → n → ℂ)
359   (hunit : ∀ i, inner (a i) (a i) = 1) :
360   trace (structureOp a) = Fintype.card ι := by
361   calc
362     trace (structureOp a)
363       = ∑ p : n, ∑ i : ι, rankOne (a i) p p := by
364         rfl
365     _ = ∑ i : ι, trace (rankOne (a i)) := by
366         rw [Finset.sum_comm]
367         rfl
368     _ = ∑ i : ι, inner (a i) (a i) := by
369         simp [trace_rankOne]
370     _ = ∑ _i : ι, (1 : ℂ) := by
371         exact Finset.sum_congr rfl (by intro i _; simp [hunit i])
372     _ = Fintype.card ι := by
373         simp
374
375 theorem rankOne_mul_rankOne_trace (u v : n → ℂ) :
376   trace (mmul (rankOne u) (rankOne v)) = inner v u * inner u v := by
377   calc
378     trace (mmul (rankOne u) (rankOne v))
379       = ∑ p : n, ∑ r : n,
380         (u p * star (u r)) * (v r * star (v p)) := by
381         rfl
382     _ = ∑ p : n, (u p * star (v p)) * ∑
383       ( r : n, star (u r) * v r) := by
384         simp_rw [Finset.mul_sum]
385         congr
386         ext p
387         apply Finset.sum_congr rfl
388         intro r _
389         ring
390     _ = ∑ ( p : n, u p * star (v p)) * inner u v := by
391         rw [Finset.sum_mul]
392         rfl
393     _ = inner v u * inner u v := by
394         congr
395         ext p

```

```

396         ring
397
398     theorem sum_rankOne_mul_rankOne_trace (a : ι → n → ℂ) : ∑
399         ( i : ι, ∑ j : ι,
400             trace (mmul (rankOne (a i)) (rankOne (a j))))
401         = ∑ i : ι, ∑ j : ι, inner (a j) (a i) * inner (a i) (a j) := by
402         simp [rankOne_mul_rankOne_trace]
403
404     end TraceIdentities
405
406     end Pseudocontexts

```